PROJECT: CREATING FUNCTIONS TO REGISTER APP USERS

You are a junior developer working in a small start-up. Your managers have asked you to develop a new account registration system for a mobile app. The system must validate user input on the sign-up form before creating an account.

The previous junior developer wrote some helper functions that validate the name, email, and password. Use these functions to register users, store their data, and implement some error handling! These have been imported into the workspace for you. They will be a great help to you when registering the user, but first you have to understand what the function does! Inspect the docstrings of each of the helper functions: `validate_name`, `validate_email` and `validate_password`.

```python
# Re-run this cell and examine the docstring of each function
from python_functions import validate_name, validate_email, validate_password,
top_level_domains

print("validate_name\n")
print(validate_name.__doc__)
print("--------------------\n")

print("validate_email\n")
print(validate_email.__doc__)
print("--------------------\n")

print("validate_password\n")
print(validate_password.__doc__)

# The top level domains variable is used in validate_email to approve only certain
email domains
print(top_level_domains)
```

```
validate_name

 Checks that the name is greater than two characters and is a string data type.

 Args:
    name (str): The inputted name from the user.

 Returns:
    bool: True if the name passes the check, False otherwise.

--------------------

validate_email

 Checks that the email address is in a valid format, has a username greater than
1 character, an '@' symbol, and an allowed domain that is in the
`top_level_domains` variable.

  Args:
    email (str): The inputted email from the user.

  Returns:
    bool: True if the email passes the checks, False otherwise.
```

```python
# Start coding here
# Use as many cells as you need
from python_functions import validate_name, validate_email, validate_password

def validate_user(name, email, password):
    """Validate the user name, email and password.

    Args:
        name (string): Name that we're attempting to validate.
        email (string): Email address that we're attempting to validate.
        password (string): Password that we're attempting to validate.

    Returns:
        bool: Returns True if all validation checks pass.

    Raises:
        ValueError: If any validation check fails.
    """
    if validate_name(name) == False:
        raise ValueError("Please make sure your name is greater than 2 characters!")

    if validate_email(email) == False:
        raise ValueError("Your email address is in the incorrect format, please
enter a valid email.")

    if validate_password(password) == False:
        raise ValueError("Your password is too weak, ensure that your password is
greater than 8 characters, "
                         "contains a capital letter and a number.")

    return True

def register_user(name, email, password):
    """Attempt to register the user if they pass validation.

    Args:
        name (string): Name of the user.
        email (string): Email address of the user.
        password (string): Password of the user.

    Returns:
        dict or bool: Returns a dictionary with the user details if validation is
successful,
        or False if the validation fails.
    """
    try:
        validate_user(name, email, password)
```

```
    except:
        return False

    user = {
        "name": name,
        "email": email,
        "password": password
    }

    return user
```